

MODULE 2 – DATA STRUCTURES AND ALGORITHMS

1. Arrays

An array is a collection of elements stored in contiguous memory locations.

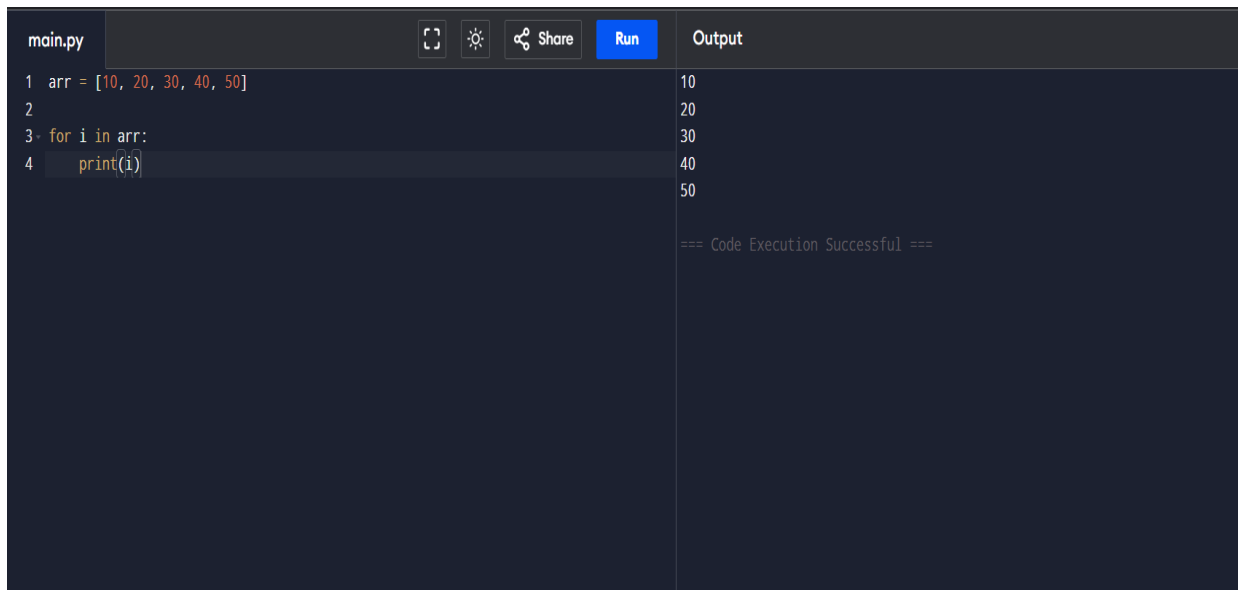
Program:

```
arr = [10, 20, 30, 40, 50]
```

```
for i in arr:
```

```
    print(i)
```

Example:



The screenshot shows a code editor with a dark theme. The editor has a tab labeled 'main.py' and a toolbar with icons for file operations, a 'Share' button, and a 'Run' button. The code in the editor is as follows:

```
1 arr = [10, 20, 30, 40, 50]
2
3 for i in arr:
4     print(i)
```

The output of the program is displayed on the right side of the editor, showing the elements of the array one by one:

```
10
20
30
40
50
```

Below the output, there is a message indicating that the code execution was successful:

```
=== Code Execution Successful ===
```

OUTPUT:

10
20
30
40
50

2. Linear Search

Linear search checks elements one by one until the target element is found.

Program:

```
arr = [10, 20, 30, 40, 50]
```

```
target = 30
```

```
for i in range(len(arr)):
    if arr[i] == target:
        print("Element found at index", i)
```

Output:

Element found at index 2

Example:

```
main.py  [ ] [ ] [ ] Share Run Output
1 arr = [10, 20, 30, 40, 50]
2
3 target = 30
4
5 for i in range(len(arr)):
6     if arr[i] == target:
7         print("Element found at index", i)
```

Element found at index 2

=== Code Execution Successful ===

3. Binary Search

Binary search works only on sorted arrays.

Program:

```
arr = [10, 20, 30, 40, 50]
```

```
low = 0
```

```
high = len(arr) - 1
```

```
target = 40
```

```
while low <= high:
```

```
    mid = (low + high) // 2
```

```
    if arr[mid] == target:
```

```
        print("Element found at index", mid)
```

```
        break
```

```
    elif arr[mid] < target:
```

low = mid + 1

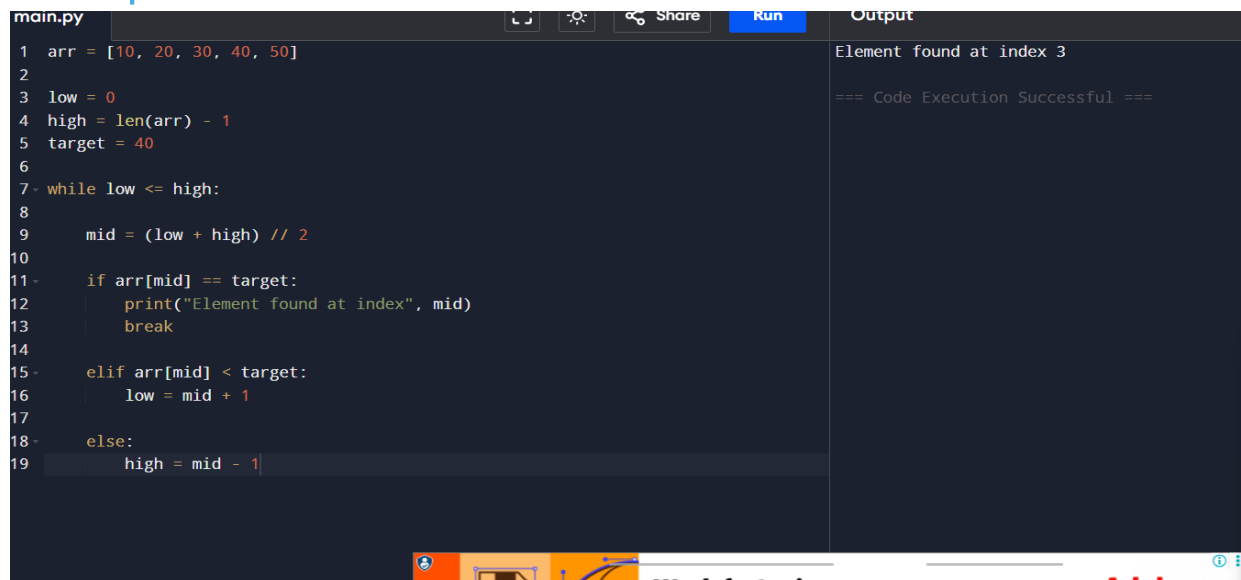
else:

high = mid - 1

Output:

Element found at index 3

Example:



```
main.py
1 arr = [10, 20, 30, 40, 50]
2
3 low = 0
4 high = len(arr) - 1
5 target = 40
6
7 while low <= high:
8
9     mid = (low + high) // 2
10
11     if arr[mid] == target:
12         print("Element found at index", mid)
13         break
14
15     elif arr[mid] < target:
16         low = mid + 1
17
18     else:
19         high = mid - 1
```

Output

Element found at index 3

=== Code Execution Successful ===

4. Bubble Sort

Bubble sort repeatedly swaps adjacent elements if they are in the wrong order.

Program:

arr = [64, 34, 25, 12, 22]

n = len(arr)

for i in range(n):

```
for j in range(0, n - i - 1):  
  
    if arr[j] > arr[j + 1]:  
  
        arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
print("Sorted array:")  
  
for i in arr:  
    print(i)
```

OUTPUT:

Sorted array:

12

22

25

34

64

MY EXAMPLE:

```
main.py [ ] [ ] [ ] Share Run Output
1 arr = [64, 34, 25, 12, 22]
2
3 n = len(arr)
4
5 for i in range(n):
6
7     for j in range(0, n - i - 1):
8
9         if arr[j] > arr[j + 1]:
10
11             arr[j], arr[j + 1] = arr[j + 1], arr[j]
12
13 print("Sorted array:")
14
15 for i in arr:
16     print(i)
```

Sorted array:
12
22
25
34
64

=== Code Execution Successful ===

5. Merge Sort

Merge sort uses the divide-and-conquer approach.

Time Complexity:

$O(n \log n)$

6. Stack

Stack follows the LIFO principle (Last In First Out).

Program:

```
stack = []
```

```
stack.append(10)
```

```
stack.append(20)
```

```
stack.append(30)
```

```
print("Stack:", stack)
```

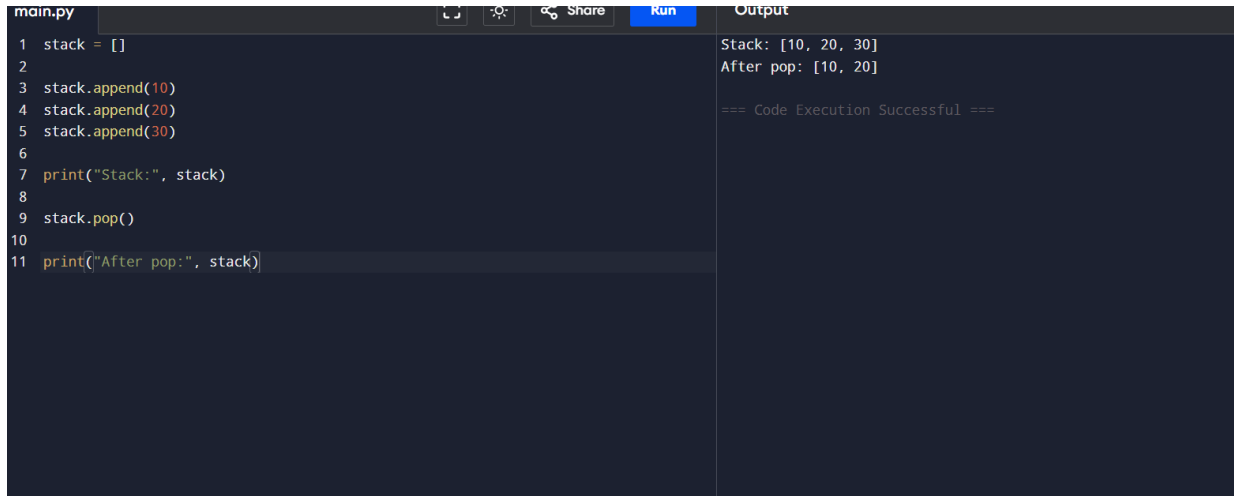
```
stack.pop()
```

```
print("After pop:", stack)
```

OUTPUT:

Stack: [10, 20, 30]

After pop: [10, 20]

A screenshot of a code editor interface. The left pane shows a file named 'main.py' with the following Python code:

```
1 stack = []
2
3 stack.append(10)
4 stack.append(20)
5 stack.append(30)
6
7 print("Stack:", stack)
8
9 stack.pop()
10
11 print("After pop:", stack)
```

The right pane, titled 'Output', displays the execution results:

```
Stack: [10, 20, 30]
After pop: [10, 20]

=== Code Execution Successful ===
```

7. Queue

Queue follows the FIFO principle (First In First Out).

Program:

```
queue = []
```

```
queue.append(10)
```

```
queue.append(20)
```

```
queue.append(30)
```

```
print("Queue:", queue)
```

```
queue.pop(0)
```

```
print("After dequeue:", queue)
```

OUTPUT:

Queue: [10, 20, 30]

After dequeue: [20, 30]

MY EXAMPLE:


```
1 queue = []
2
3 queue.append(10)
4 queue.append(20)
5 queue.append(30)
6
7 print("Queue:", queue)
8
9 queue.pop(0)
10
11 print("After dequeue:", queue)
```

```
Queue: [10, 20, 30]
After dequeue: [20, 30]
```

```
=== Code Execution Successful ===
```

Conclusion:

Data structures and algorithms such as arrays, searching techniques, sorting algorithms, stack, and queue were studied and implemented using Python. These concepts help in writing efficient and optimized programs.

